Architecture 1001: x86-64 Assembly SingleLocalVariable.c

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

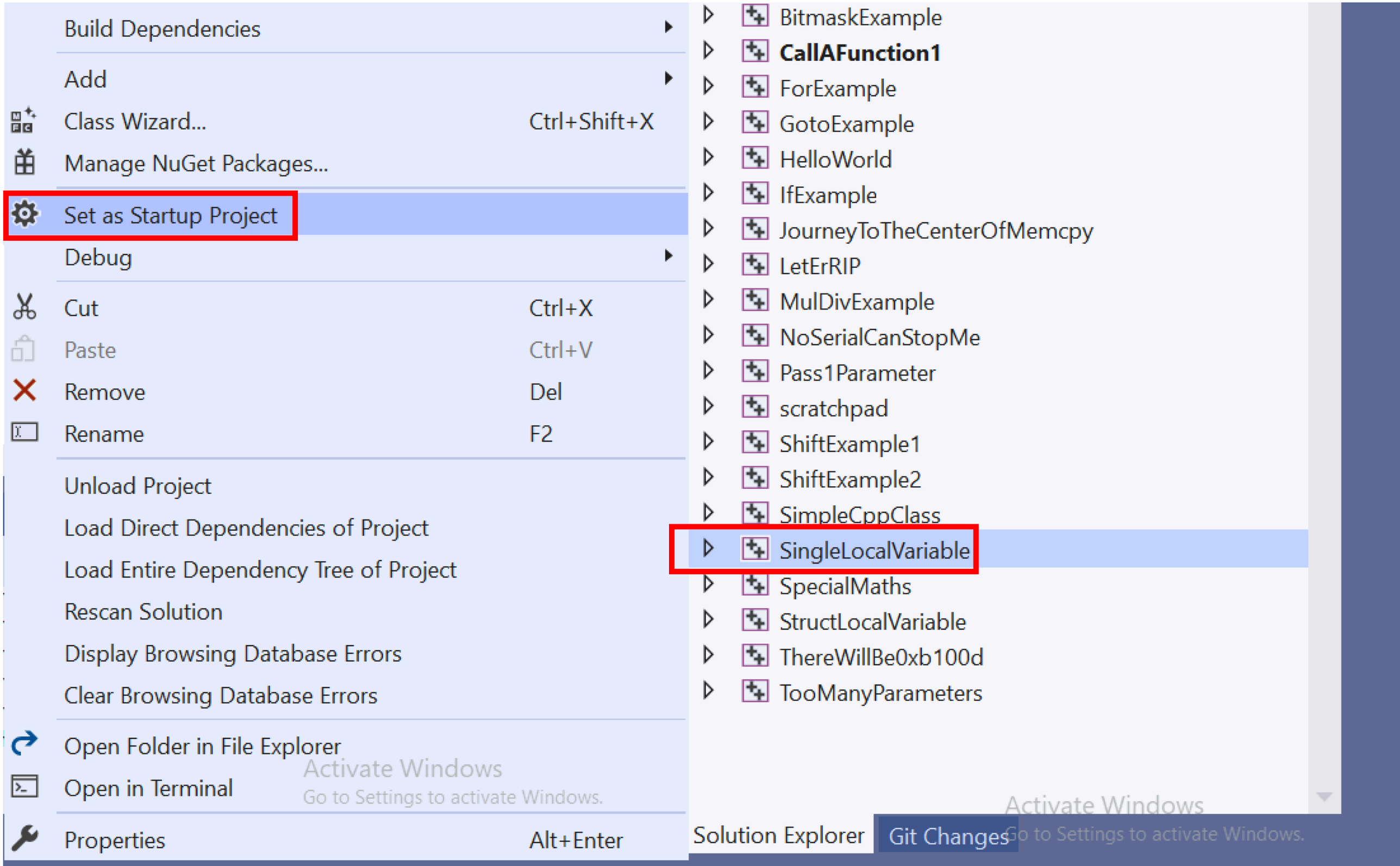
SingleLocalVariable.c

Adding a single local variable

```c
int func(){
    int i = 0x5ca1ab1e;
    return i;
}
int main(){
    return func();
}
```

```
func:
0000000140001000  sub          rsp,18h
0000000140001004  mov          dword ptr [rsp],5CA1AB1Eh
000000014000100B  mov          eax,dword ptr [rsp]
000000014000100E  add          rsp,18h
0000000140001012  ret
main:
0000000140001020  sub          rsp,28h
0000000140001024  call         func (0140001000h)
0000000140001029  add          rsp,28h
000000014000102D  ret
```

Right click on the next project and select "Set as StartUp Project" as you move between class examples to make them launch when you run the debugger. Its text will go **bold** after.



SingleLocalVariable.c Stack

- If we step *through* line 0000000140001004 in func(), the stack looks like this:

00000000`0014FE08	
00000000`0014FE00	
00000000`0014FDF8	
00000000`0014FDF0	
00000000`0014FDE8	
00000000`0014FDE0	
00000000`0014FDD8	
00000000`0014FDD0	
00000000`0014FDC8	
00000000`0014FDC0	



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!



STOP

&&

STEP THROUGH

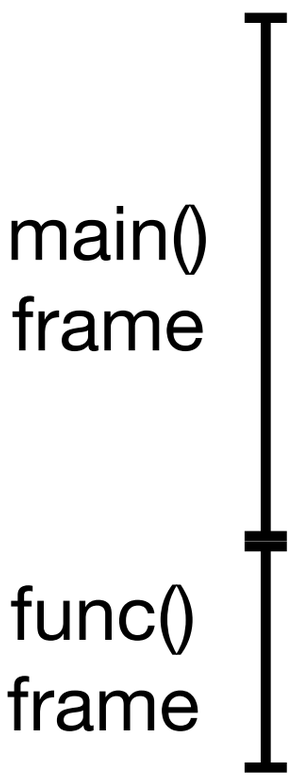
THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!

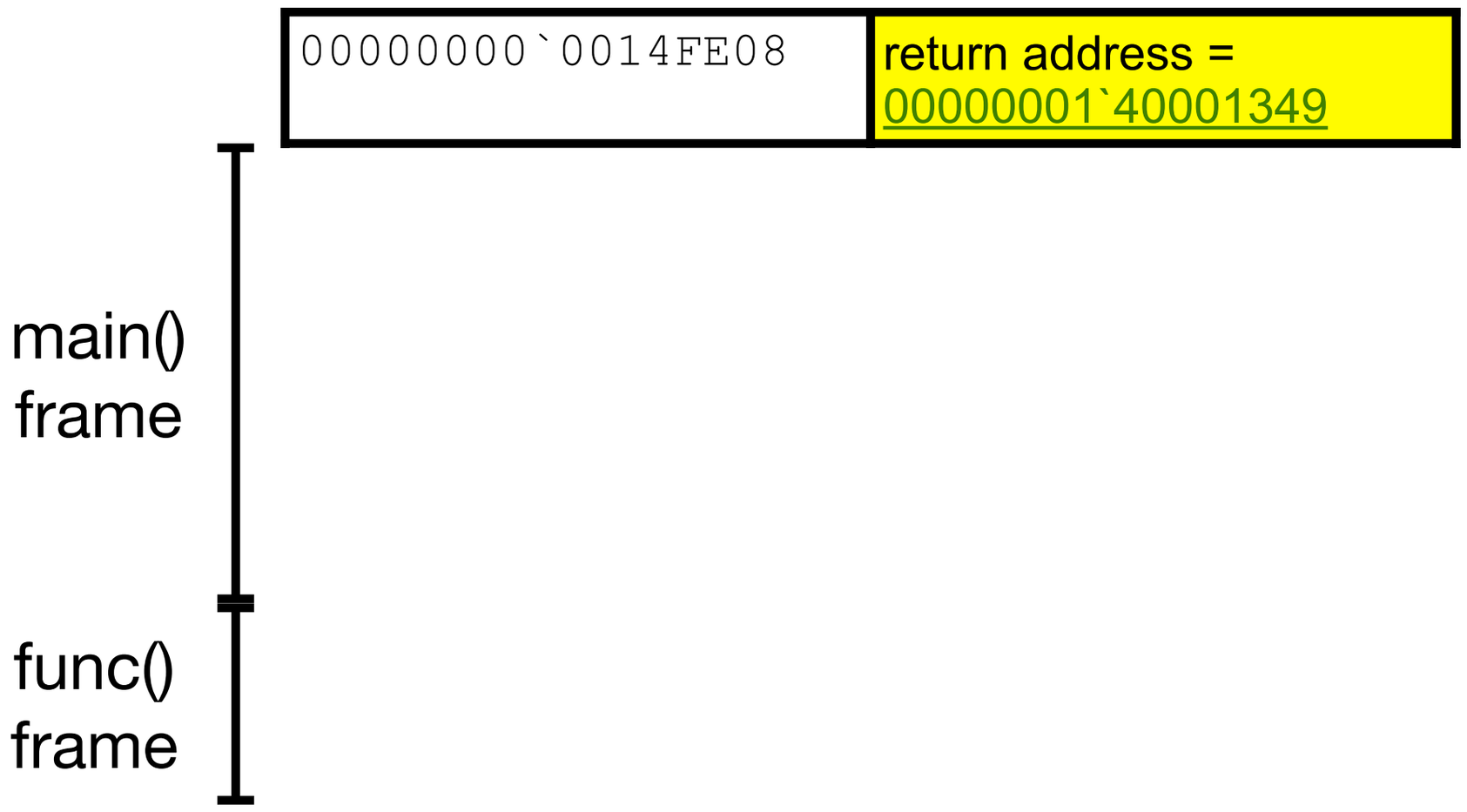
SingleLocalVariable.c Stack

- If we step *through* line 0000000140001004 in func(), the stack looks like this:



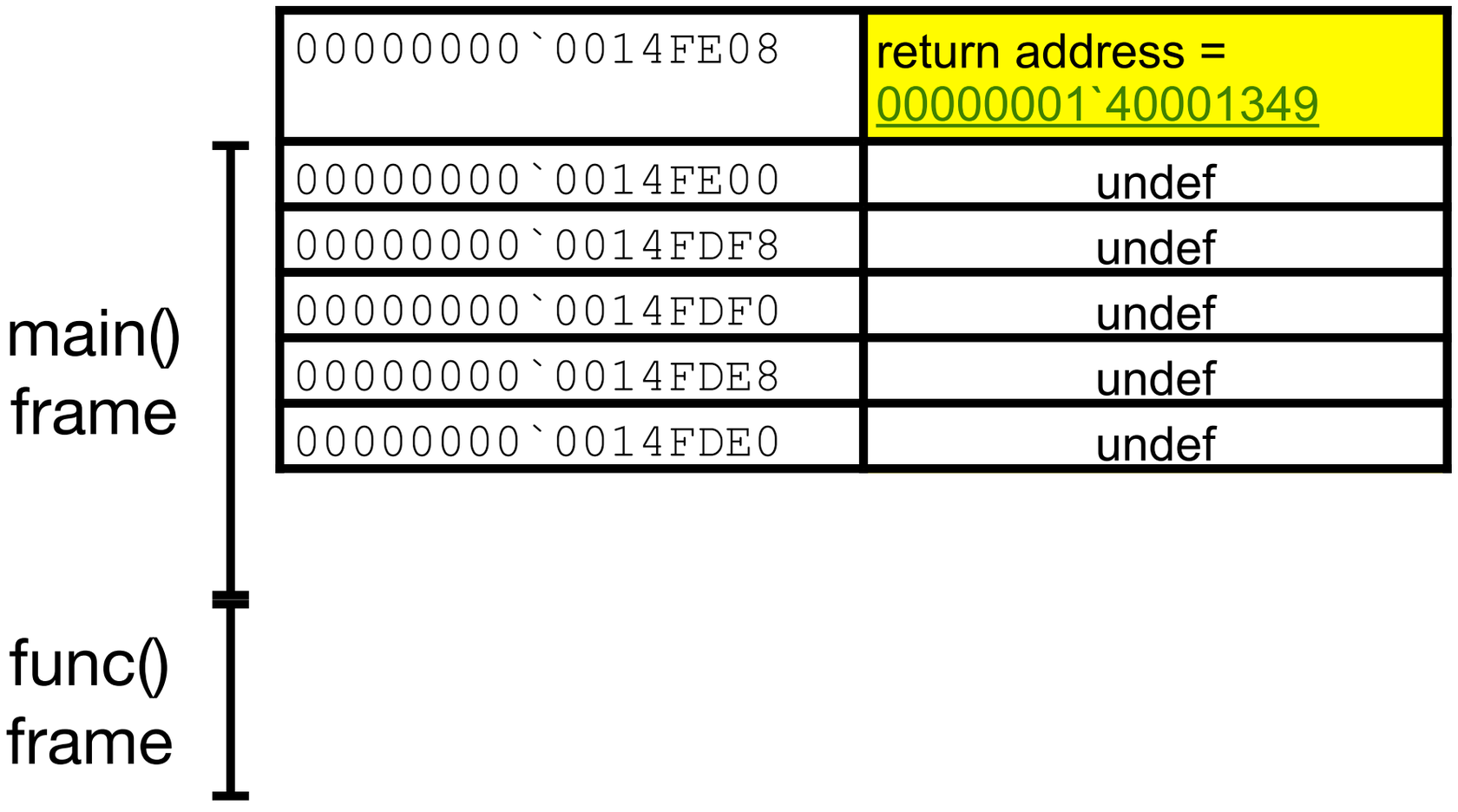
SingleLocalVariable.c Stack

- If we step *through* line 0000000140001004 in func(), the stack looks like this:



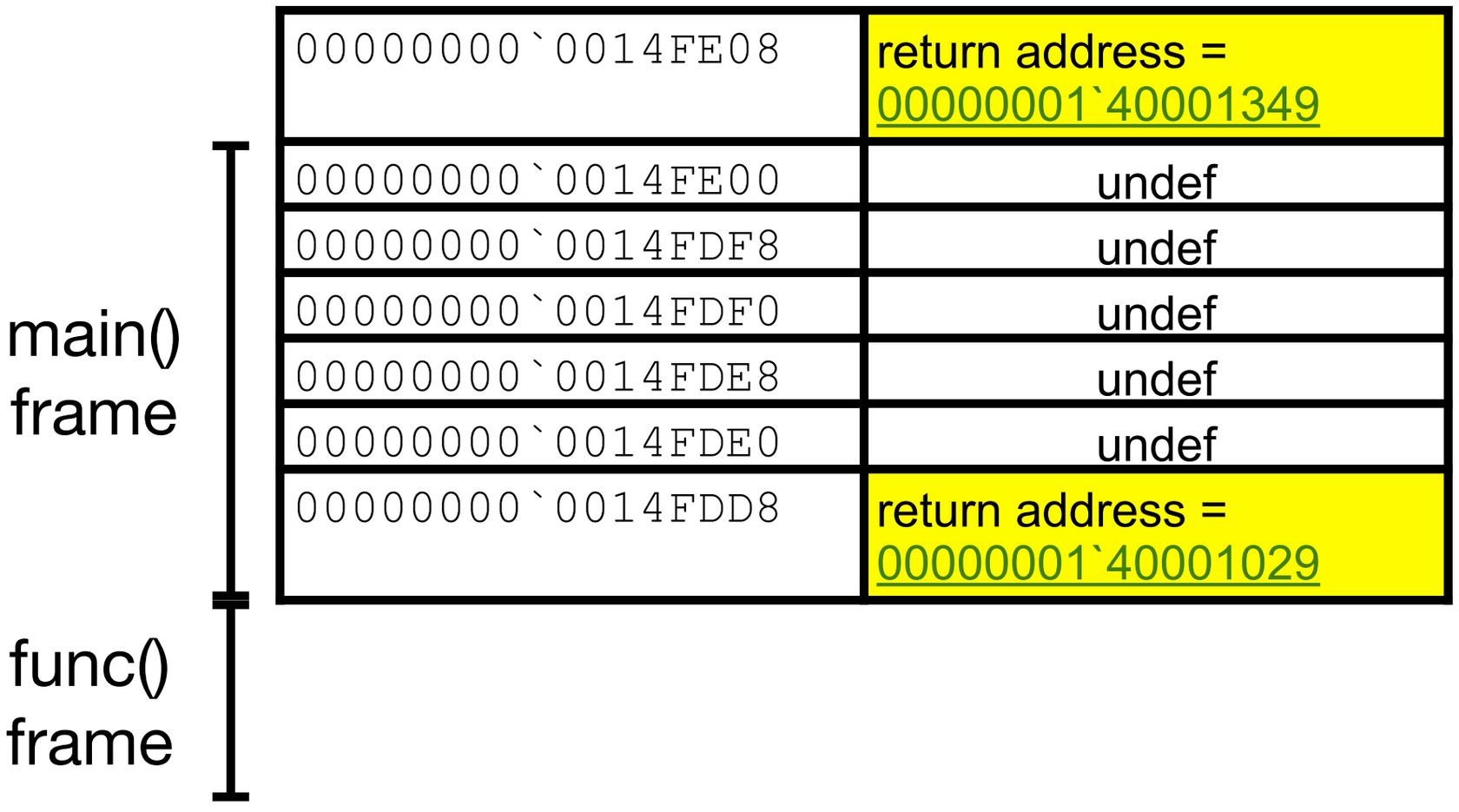
SingleLocalVariable.c Stack

- If we step *through* line 0000000140001004 in func(), the stack looks like this:



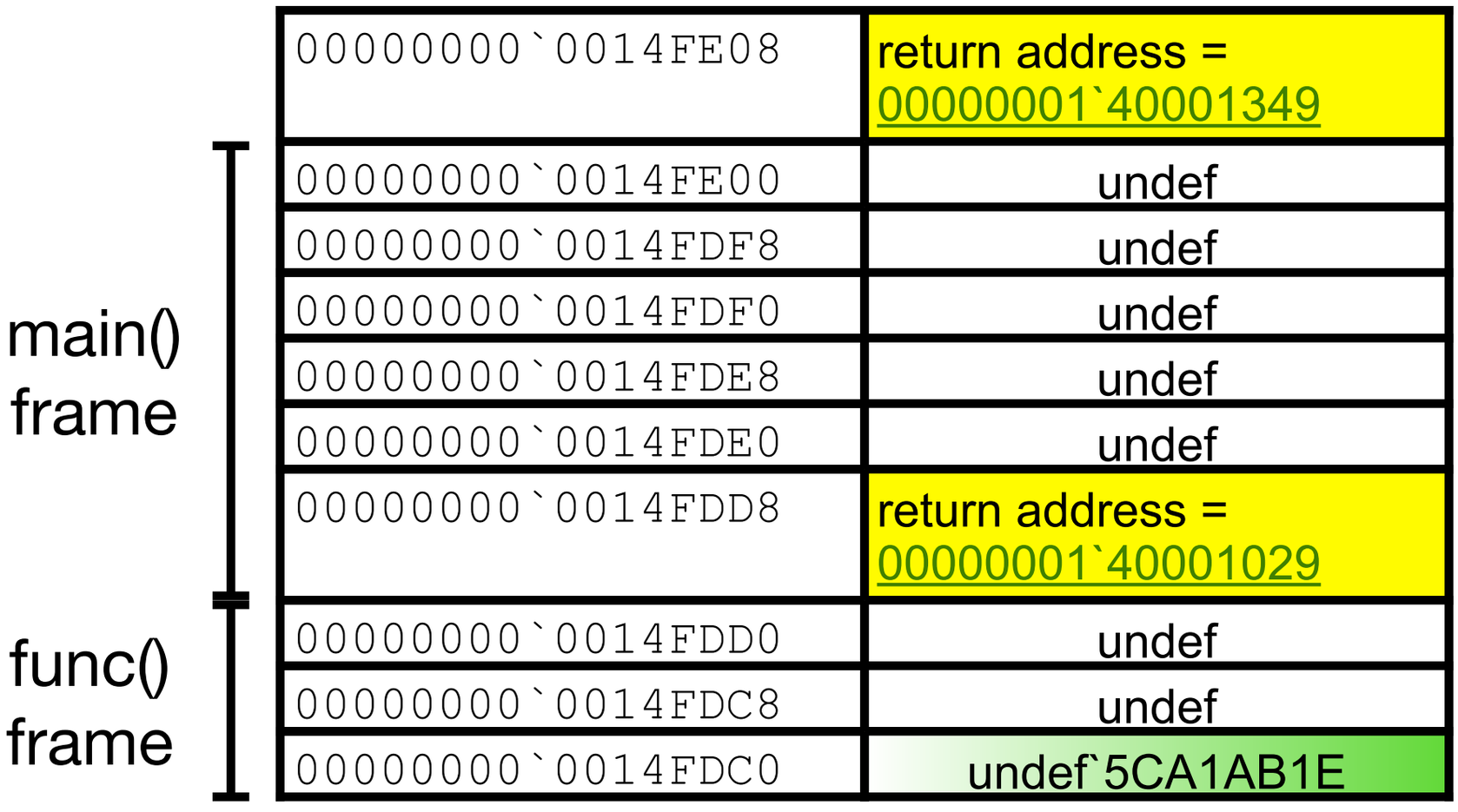
SingleLocalVariable.c Stack

- If we step *through* line 0000000140001004 in func(), the stack looks like this:



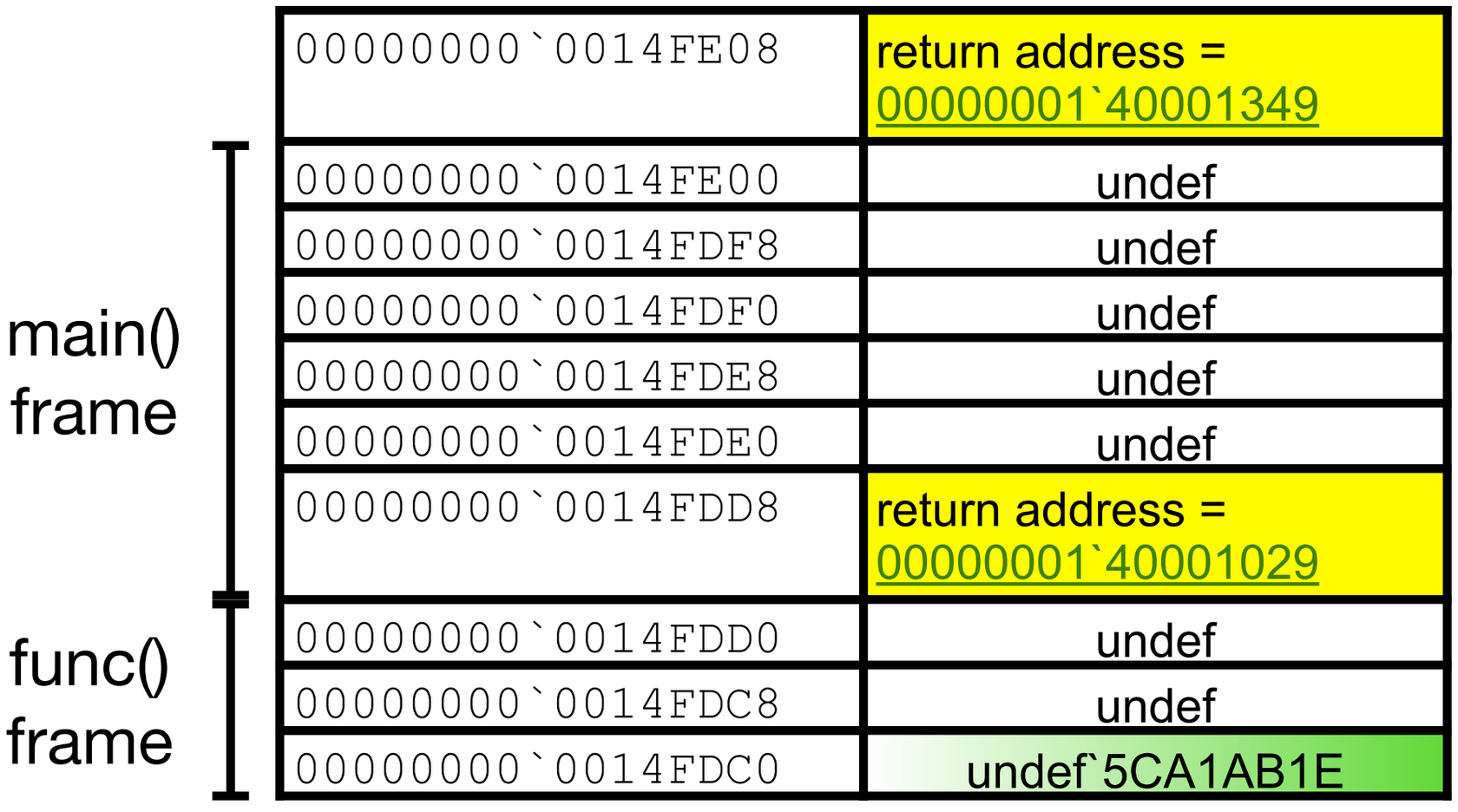
SingleLocalVariable.c Stack

- If we step *through* line 0000000140001004 in func(), the stack looks like this:



SingleLocalVariable.c Stack

- If we step *through* line 0000000140001004 in func(), the stack looks like this:



A Little Bit About r/mX Forms That I Skipped

- In Intel Syntax, for r/mX memory descriptions, it will use things like "qword ptr", "dword ptr", or "word ptr" to indicate the size of the data being operated on (8, 4, and 2 bytes respectively)

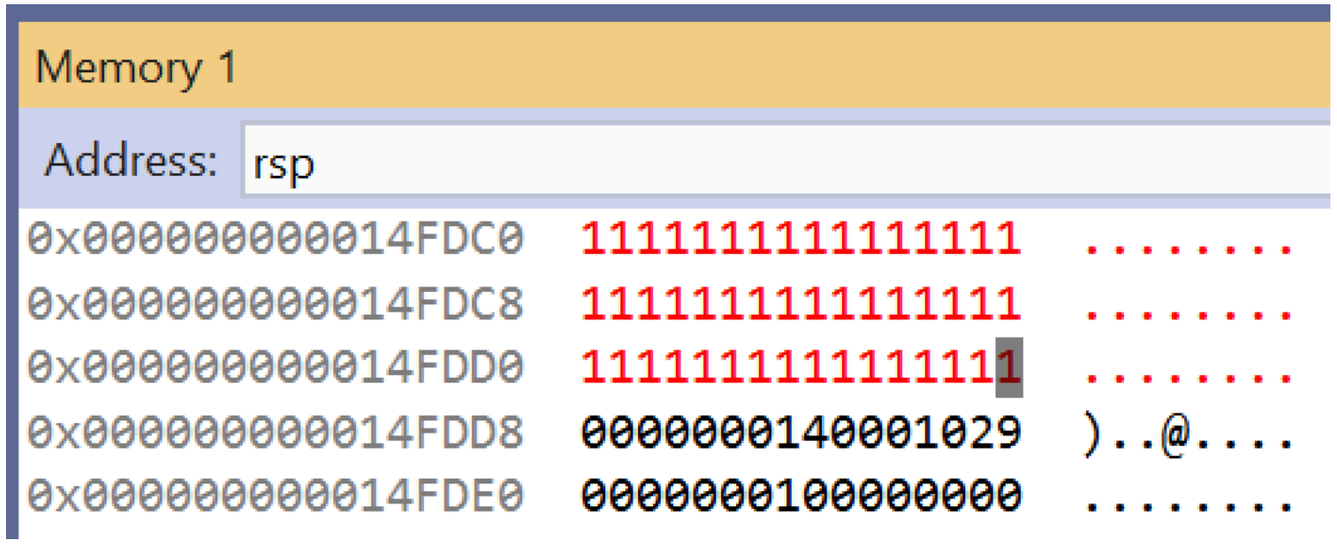
```
mov    qword ptr [rsp+10h],rax

mov    dword ptr [rsp],5CA1AB1Eh

mov    word ptr [rsp],ax
```

An Easy Way to See What Memory Changes

- You can click into the VS memory window and edit memory to initialize uninitialized memory, and then see what actually changes



SingleLocalVariable.c Takeaways

- Local variables lead to an allocation of space on the stack, within the function where the variable is scoped to
- In VS there is an over-allocation of space for local variables
  - 0x18 reserved for only 0x4 (int) worth of data

```
int func(){
    int i = 0x5ca1ab1e;
    return i;
}
int main(){
    return func();
}
```

```
func:
0000000140001000  sub          rsp,18h
0000000140001004  mov          dword ptr [rsp],5CA1AB1Eh
000000014000100B  mov          eax,dword ptr [rsp]
000000014000100E  add          rsp,18h
0000000140001012  ret
main:
0000000140001020  sub          rsp,28h
0000000140001024  call         func (0140001000h)
0000000140001029  add          rsp,28h
000000014000102D  ret
```

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?
- Why is VS over-allocating space for a single local variable?

Simple Stack Diagram 3

stack bottom

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

stack top

Simple Stack Diagram 3

stack bottom

"return address" **to return from main() to invoke_main()**

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

stack top

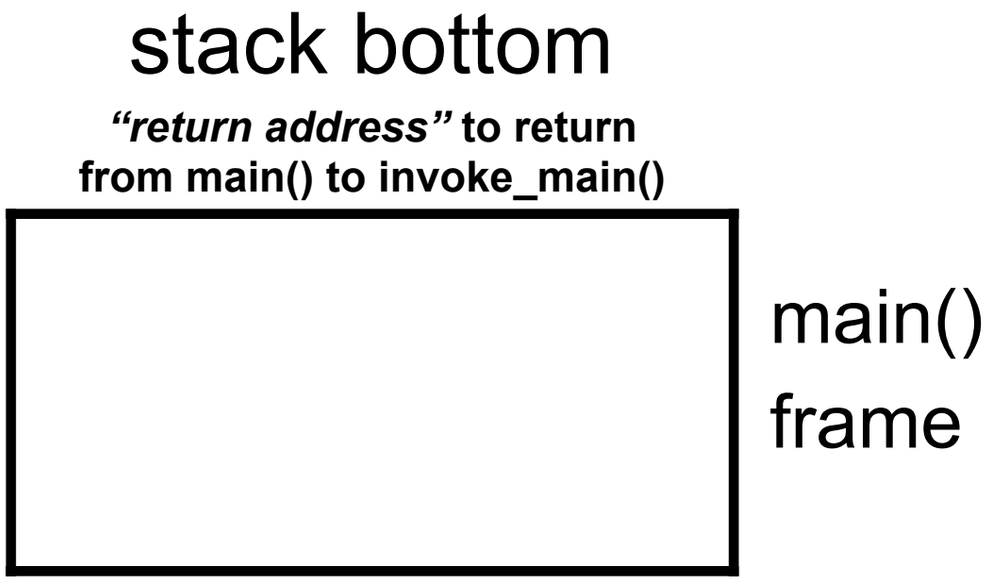
Simple Stack Diagram 3

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}
```

```c
int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



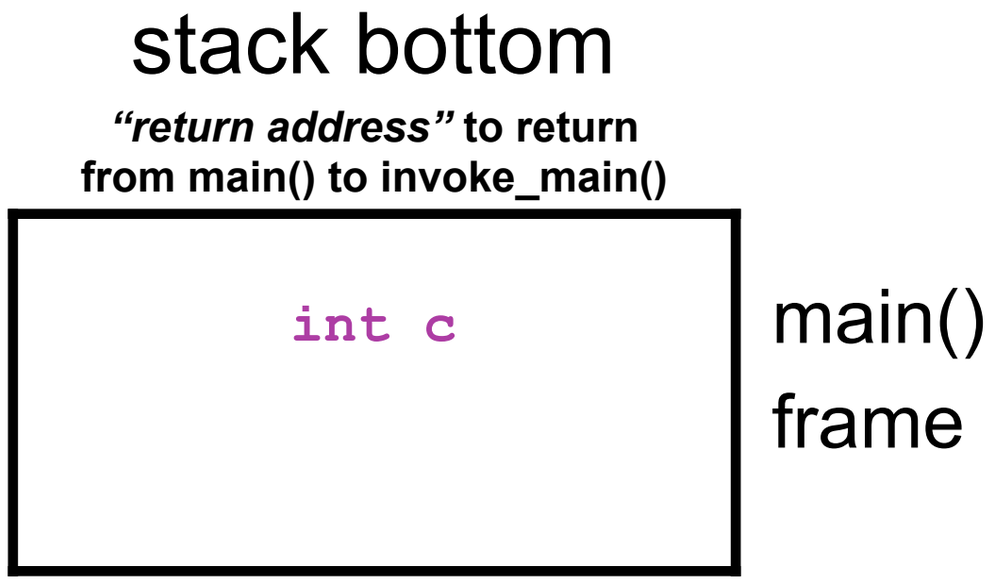
Simple Stack Diagram 3

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



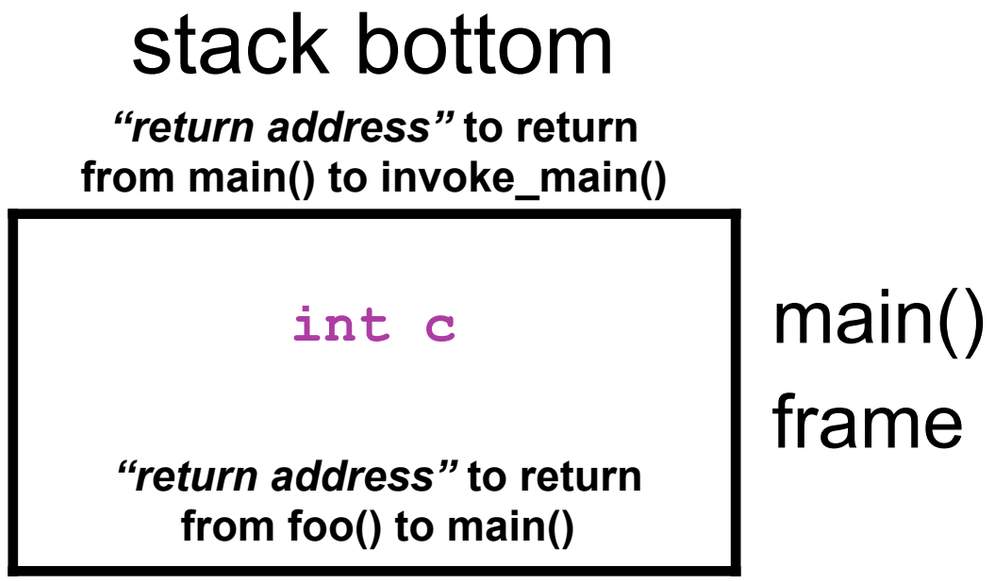
Simple Stack Diagram 3

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



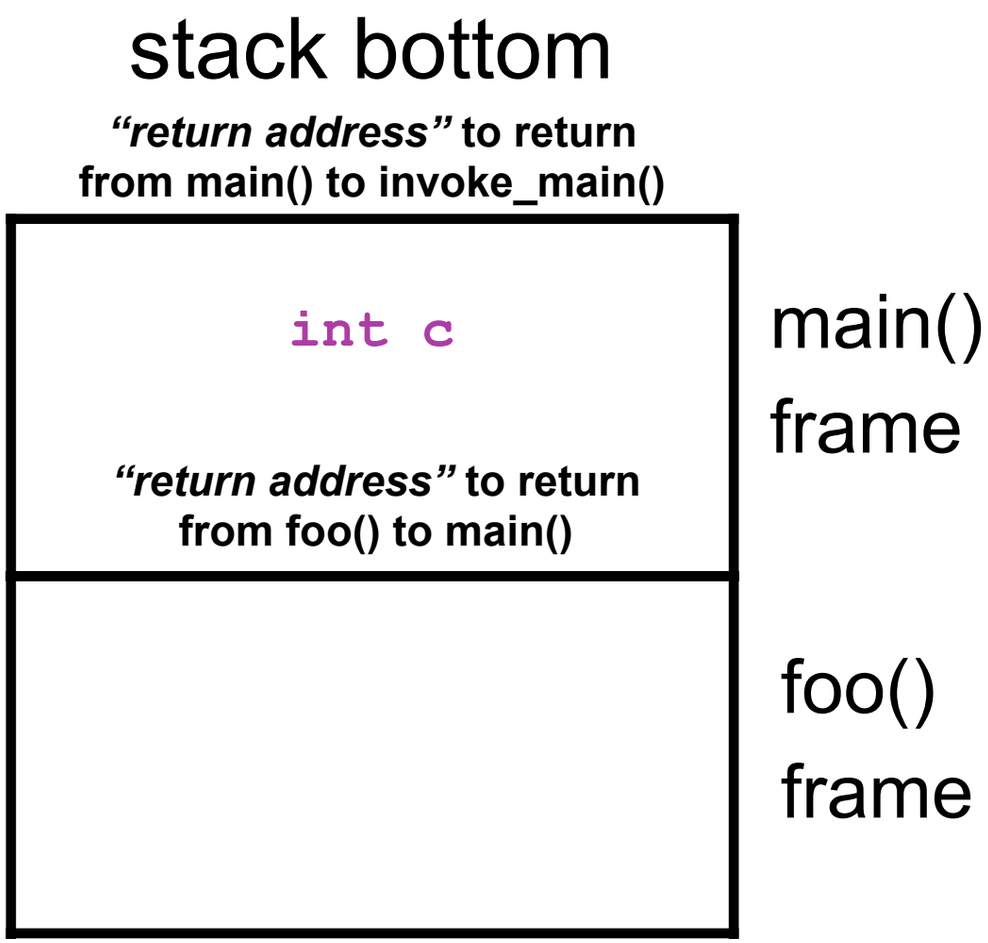
Simple Stack Diagram 3

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



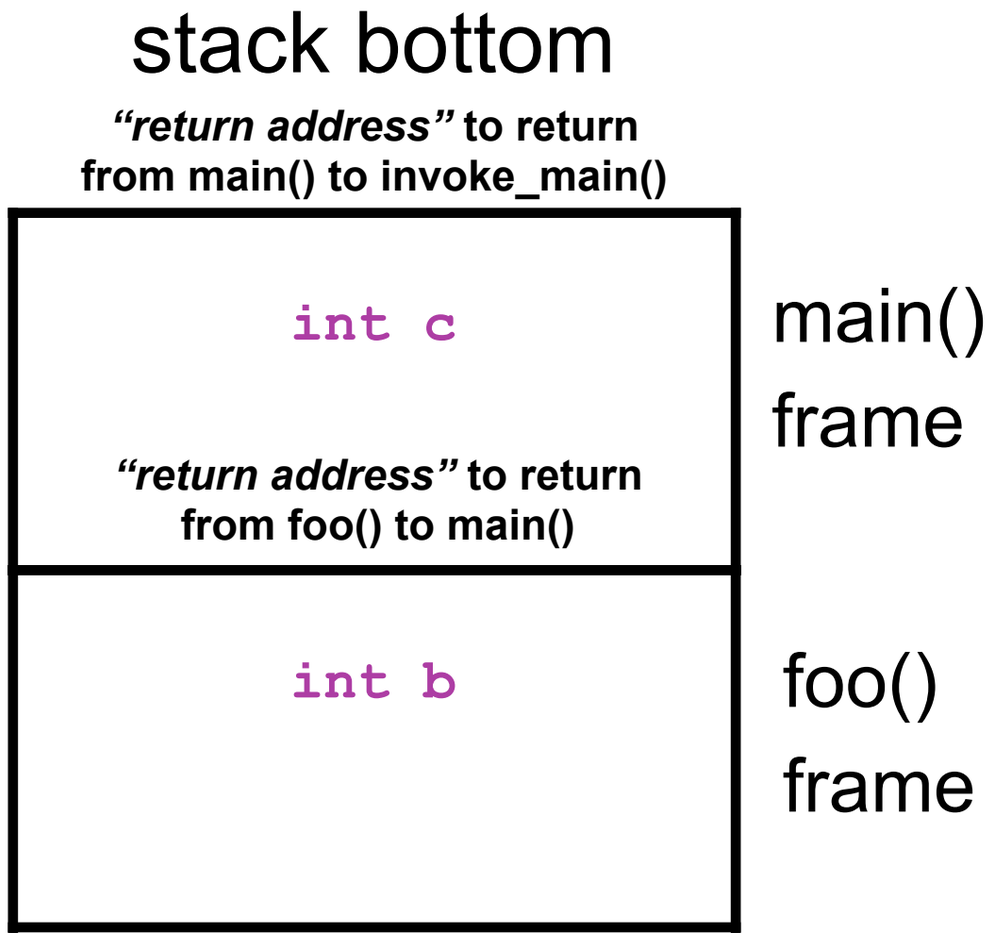
Simple Stack Diagram 3

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



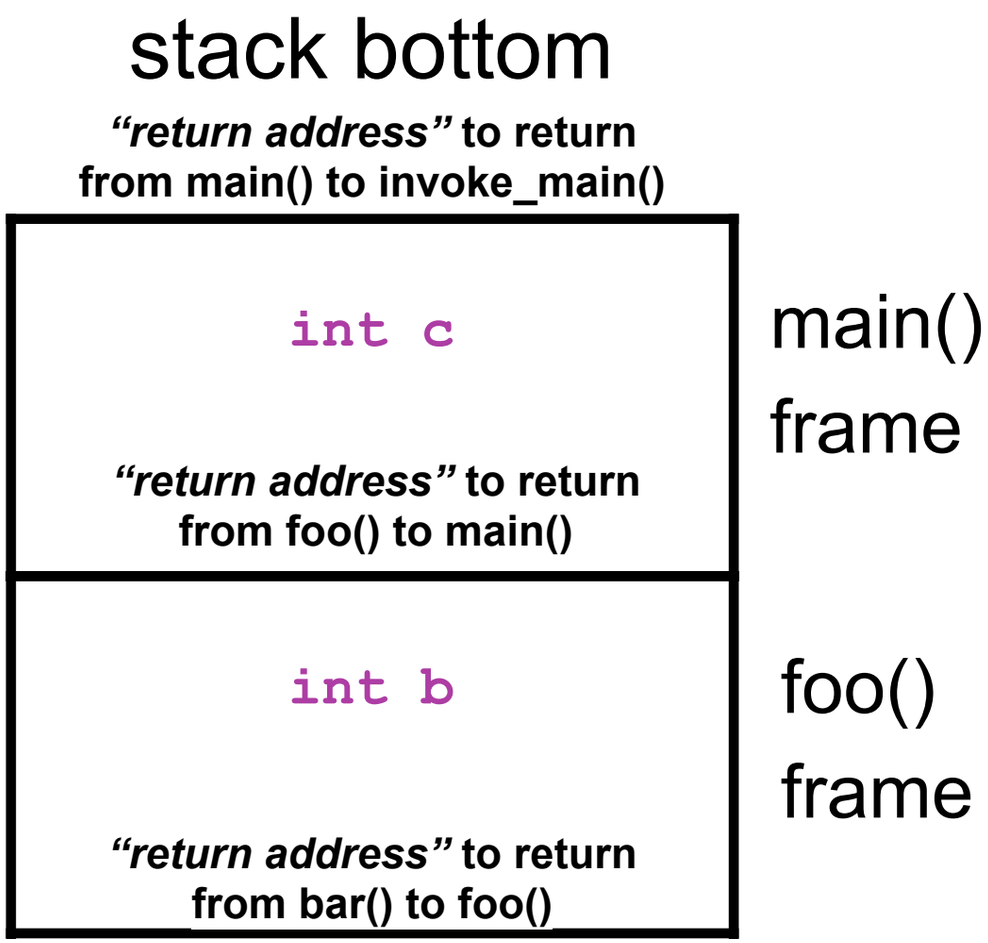
Simple Stack Diagram 3

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



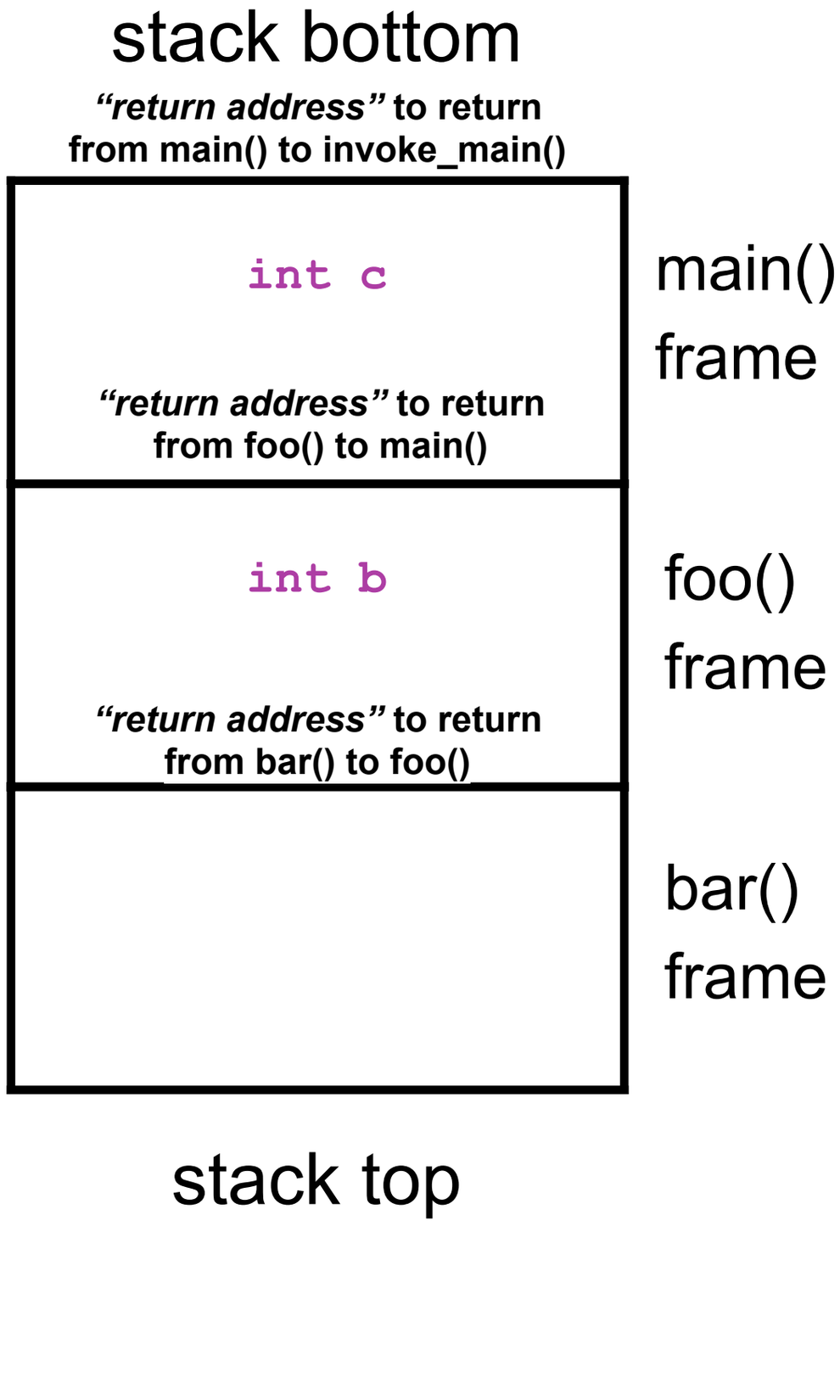
Simple Stack Diagram 3

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



Simple Stack Diagram 3

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

